Appendix F. Using the GDB Debugger

By the time you read this appendix, you will likely have written at least one program with an error in it. In assembly language, even minor errors usually have results such as the whole program crashing with a segmentation fault error. In most programming languages, you can simply print out the values in your variables as you go along, and use that output to find out where you went wrong. In assembly language, calling output functions is not so easy. Therefore, to aid in determining the source of errors, you must use a *source debugger*.

A debugger is a program that helps you find bugs by stepping through the program one step at a time, letting you examine memory and register contents along the way. A *source debugger* is a debugger that allows you to tie the debugging operation directly to the source code of a program. This means that the debugger allows you to look at the source code as you typed it in - complete with symbols, labels, and comments.

The debugger we will be looking at is GDB - the GNU Debugger. This application is present on almost all GNU/Linux distributions. It can debug programs in multiple programming languages, including assembly language.

An Example Debugging Session

The best way to explain how a debugger works is by using it. The program we will be using the debugger on is the `maximum` program used in Chapter 3. Let's say that you entered the program perfectly, except that you left out the line:

```
incl %edi
```

When you run the program, it just goes in an infinite loop - it never exits. To determine the cause, you need to run the program under GDB. However, to do this, you need to have the assembler include debugging information in the executable. All you need to do to enable this is to add the `--gstabs` option to the `as` command. So, you would assemble it like this:

```
as --gstabs maximum.s -o maximum.o
```

Linking would be the same as normal. "stabs" is the debugging format used by GDB. Now, to run the program under the debugger, you would type in `gdb ./maximum`. Be sure that the source files are in the current directory. The output should look similar to this:

```
GNU gdb Red Hat Linux (5.2.1-4)
Copyright 2002 Free Software Foundation, Inc.
GDB is free software, covered by the GNU General Public
License, and you are welcome to change it and/or
distribute copies of it under certain conditions. Type
"show copying" to see the conditions.  There is
absolutely no warranty for GDB.  Type "show warranty"
for details.
This GDB was configured as "i386-redhat-linux"...
(gdb)
```

Depending on which version of GDB you are running, this output may vary slightly. At this point, the program is loaded, but is not running yet. The debugger is waiting your command. To run your program, just type in `run`. This will not return, because the program is running in an infinite loop. To stop the program, hit control-c. The screen will then say this:

```
Starting program: /home/johnnyb/maximum

Program received signal SIGINT, Interrupt.
start_loop () at maximum.s:34
34              movl data_items(,%edi,4), %eax
Current language:  auto; currently asm
(gdb)
```

This tells you that the program was interrupted by the SIGINT signal (from your control-c), and was within the section labelled `start_loop`, and was executing on line 34 when it stopped. It gives you the code that it is about to execute.

Depending on exactly when you hit control-c, it may have stopped on a different line or a different instruction than the example.

One of the best ways to find bugs in a program is to follow the flow of the program to see where it is branching incorrectly. To follow the flow of this program, keep on entering `stepi` (for "step instruction"), which will cause the computer to execute one instruction at a time. If you do this several times, your output will look something like this:

```
(gdb) stepi
35              cmpl %ebx, %eax
(gdb) stepi
36              jle start_loop
(gdb) stepi
32              cmpl $0, %eax
(gdb) stepi
33              je loop_exit
(gdb) stepi
34              movl data_items(,%edi,4), %eax
(gdb) stepi
35              cmpl %ebx, %eax
(gdb) stepi
36              jle start_loop
(gdb) step
32              cmpl $0, %eax
```

As you can tell, it has looped. In general, this is good, since we wrote it to loop. However, the problem is that it is *never stopping*. Therefore, to find out what the problem is, let's look at the point in our code where we should be exitting the loop:

```
cmpl  $0, %eax
je    loop_exit
```

Basically, it is checking to see if `%eax` hits zero. If so, it should exit the loop. There are several things to check here. First of all, you may have left this piece out

altogether. It is not uncommon for a programmer to forget to include a way to exit a loop. However, this is not the case here. Second, you should make sure that `loop_exit` actually is outside the loop. If we put the label in the wrong place, strange things would happen. However, again, this is not the case.

Neither of those potential problems are the culprit. So, the next option is that perhaps `%eax` has the wrong value. There are two ways to check the contents of register in GDB. The first one is the command `info register`. This will display the contents of all registers in hexadecimal. However, we are only interested in `%eax` at this point. To just display `%eax` we can do `print/$eax` to print it in hexadecimal, or do `print/d $eax` to print it in decimal. Notice that in GDB, registers are prefixed with dollar signs rather than percent signs. Your screen should have this on it:

```
(gdb) print/d $eax
$1 = 3
(gdb)
```

This means that the result of your first inquiry is 3. Every inquiry you make will be assigned a number prefixed with a dollar sign. Now, if you look back into the code, you will find that 3 is the first number in the list of numbers to search through. If you step through the loop a few more times, you will find that in every loop iteration `%eax` has the number 3. This is not what should be happening. `%eax` should go to the next value in the list in every iteration.

Okay, now we know that `%eax` is being loaded with the same value over and over again. Let's search to see where `%eax` is being loaded from. The line of code is this:

```
 movl data_items(,%edi,4), %eax
```

So, step until this line of code is ready to execute. Now, this code depends on two values - `data_items` and `%edi`. `data_items` is a symbol, and therefore constant. It's a good idea to check your source code to make sure the label is in

front of the right data, but in our case it is. Therefore, we need to look at `%edi`. So, we need to print it out. It will look like this:

```
(gdb) print/d $edi
$2 = 0
(gdb)
```

This indicates that `%edi` is set to zero, which is why it keeps on loading the first element of the array. This should cause you to ask yourself two questions - what is the purpose of `%edi`, and how should its value be changed? To answer the first question, we just need to look in the comments. `%edi` is holding the current index of `data_items`. Since our search is a sequential search through the list of numbers in `data_items`, it would make sense that `%edi` should be incremented with every loop iteration.

Scanning the code, there is no code which alters `%edi` at all. Therefore, we should add a line to increment `%edi` at the beginning of every loop iteration. This happens to be exactly the line we tossed out at the beginning. Assembling, linking, and running the program again will show that it now works correctly.

Hopefully this exercise provided some insight into using GDB to help you find errors in your programs.

Breakpoints and Other GDB Features

The program we entered in the last section had an infinite loop, and could be easily stopped using control-c. Other programs may simply abort or finish with errors. In these cases, control-c doesn't help, because by the time you press control-c, the program is already finished. To fix this, you need to set *breakpoints*. A breakpoint is a place in the source code that you have marked to indicate to the debugger that it should stop the program when it hits that point.

To set breakpoints you have to set them up before you run the program. Before issuing the `run` command, you can set up breakpoints using the `break` command.

For example, to break on line 27, issue the command `break 27`. Then, when the program crosses line 27, it will stop running, and print out the current line and instruction. You can then step through the program from that point and examine registers and memory. To look at the lines and line numbers of your program, you can simply use the command `l`. This will print out your program with line numbers a screen at a time.

When dealing with functions, you can also break on the function names. For example, in the factorial program in Chapter 4, we could set a breakpoint for the factorial function by typing in `break factorial`. This will cause the debugger to break immediately after the function call and the function setup (it skips the pushing of `%ebp` and the copying of `%esp`).

When stepping through code, you often don't want to have to step through every instruction of every function. Well-tested functions are usually a waste of time to step through except on rare occasion. Therefore, if you use the `nexti` command instead of the `stepi` command, GDB will wait until completion of the function before going on. Otherwise, with `stepi`, GDB would step you through every instruction within every called function.

> **Warning**
>
> One problem that GDB has is with handling interrupts. Often times GDB will miss the instruction that immediately follows an interrupt. The instruction is actually executed, but GDB doesn't step through it. This should not be a problem - just be aware that it may happen.

GDB Quick-Reference

This quick-reference table is copyright 2002 Robert M. Dondero, Jr., and is used by permission in this book. Parameters listed in brackets are optional.

Table F-1. Common GDB Debugging Commands

Miscellaneous	
quit	Exit GDB
help [cmd]	Print description of debugger command `cmd`. Without `cmd`, prints a list of topics.
directory [dir1] [dir2] ...	Add directories `dir1`, `dir2`, etc. to the list of directories searched for source files.
Running the Program	
run [arg1] [arg2] ...	Run the program with command line arguments `arg1`, `arg2`, etc.
set args arg1 [arg2] ...	Set the program's command-line arguments to `arg1`, `arg2`, etc.
show args	Print the program's command-line arguments.
Using Breakpoints	
info breakpoints	Print a list of all breakpoints and their numbers (breakpoint numbers are used for other breakpoint commands).
break *linenum*	Set a breakpoint at line number *linenum*.
break **addr*	Set a breakpoint at memory address *addr*.
break *fn*	Set a breakpoint at the beginning of function *fn*.
condition *bpnum expr*	Break at breakpoint *bpnum* only if expression *expr* is non-zero.

Using Breakpoints	
command [*bpnum*] *cmd1* [*cmd2*] ...	Execute commands *cmd1*, *cmd2*, etc. whenever breakpoint *bpnum* (or the current breakpoint) is hit.
continue	Continue executing the program.
kill	Stop executing the program.
delete [*bpnum1*] [*bpnum2*] ...	Delete breakpoints *bpnum1*, *bpnum2*, etc., or all breakpoints if none specified.
clear **addr*	Clear the breakpoint at memory address *addr*.
clear [*fn*]	Clear the breakpoint at function *fn*, or the current breakpoint.
clear *linenum*	Clear the breakpoint at line number *linenum*.
disable [*bpnum1*] [*bpnum2*] ...	Disable breakpoints *bpnum1*, *bpnum2*, etc., or all breakpoints if none specified.
enable [*bpnum1*] [*bpnum2*] ...	Enable breakpoints *bpnum1*, *bpnum2*, etc., or all breakpoints if none specified.
Stepping through the Program	
nexti	"Step over" the next instruction (doesn’t follow function calls).
stepi	"Step into" the next instruction (follows function calls).
finish	"Step out" of the current function.
Examining Registers and Memory	
info registers	Print the contents of all registers.

Examining Registers and Memory	
print/*f* $*reg*	Print the contents of register *reg* using format *f*. The format can be x (hexadecimal), u (unsigned decimal), o (octal), a(address), c (character), or f (floating point).
x/*rsf addr*	Print the contents of memory address *addr* using repeat count *r*, size *s*, and format *f*. Repeat count defaults to 1 if not specified. Size can be b (byte), h (halfword), w (word), or g (double word). Size defaults to word if not specified. Format is the same as for print, with the additions of s (string) and i (instruction).
info display	Shows a numbered list of expressions set up to display automatically at each break.
display/*f* $*reg*	At each break, print the contents of register *reg* using format *f*.
display/*s*i addr	At each break, print the contents of memory address *addr* using size *s* (same options as for the x command).
display/*s*s addr	At each break, print the string of size *s* that begins in memory address *addr*.
undisplay *displaynum*	Remove *displaynum* from the display list.
Examining the Call Stack	
where	Print the call stack.
backtrace	Print the call stack.

Examining the Call Stack	
frame	Print the top of the call stack.
up	Move the context toward the bottom of the call stack.
down	Move the context toward the top of the call stack.